

UniSketch: A Unified Sketch for Efficient General Flow Spread Measurement

Supplement Material

A Algorithm of Per-packet Processing

Algorithm 1: Per-packet Processing

Input: A stream of flow elements $(f_1, e_1), (f_2, e_2), \dots$

- 1 Initialize all sketch fields and n_s to 0, set p_M to $1 - 2^{-l}$;
- 2 **foreach** flow element (f, e) in the network stream **do**
- 3 compute layer index $i = \phi(H_v(f \oplus e))$;
- 4 compute column index
- 5 $j = H_m(f \oplus R[H_s(f \oplus e) \bmod s]) \bmod m$;
- 6 **if** $i < l$ and $M[i][j] == 0$ **then**
- 7 $M[i][j] \leftarrow 1$;
- 8 compute bucket index $u = H_{m'}(f) \bmod m'$;
- 9 **if** flow f in $B[u]$ **then**
- 10 $\delta = VM(B[u].C_f, n_s, i)$; /*Case 1*/
- 11 add δ to $B[u].C_f$;
- 12 **else if** $B[u]$ has an empty slot **then**
- 13 $\delta = VM(0, n_s, i)$; /*Case 2*/
- 14 insert (f, δ) into $B[u]$;
- 15 **else**
- 16 find the smallest flow f_{tmp} in $B[u]$;
- 17 /*Case 3*/
- 18 $\delta = VM(0, n_s, i)$;
- 19 generate a random value $p \in [0, 1]$;
- 20 **if** $p < \frac{\delta}{C_{tmp}}$ **then**
- 21 replace (f_{tmp}, C_{tmp}) with (f, C_{tmp}) ;
- 22 **Function** $VM(n_f, n_s, i)$:
- 23 $\gamma = \lceil \log_2 \max\{\frac{n_f + n_s}{\rho s}, 1\} \rceil$; $\delta = 0$;
- 24 **if** $\gamma \leq i < \gamma + g$ **then**
- 25 $x = \frac{n_f + n_s}{2^{\gamma + 1} s}$;
- 26 compute $p(n_f + n_s, s, g)$ according to Equation (2);
- 27 $\delta = \frac{1}{p(n_f + n_s, s, g)}$;
- 28 update p_M and n_s according to § 3.4;
- 29 **return** δ ;

B Data Plane Approximation of Parameters

Figure 1 presents the linear fitting parameters for four commonly used values of s , where the nodes represent sample points computed according to Algorithm 1, and the straight line denotes the fitted model.

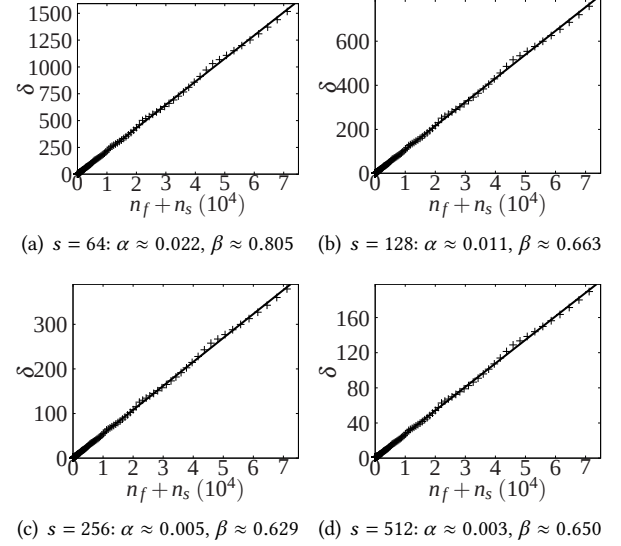


Figure 1. Linear fitting of $n_f + n_s$ and δ .

C Theoretical Analysis of UniSketch

C.1 Error Bound of MV2B

In the MV2B structure, each flow f randomly selects s columns from the m columns of bit array M , forming a V2B denoted by V_f . We can estimate its spread by $\hat{n}_f = \hat{n} - \hat{n}_s$, where $\hat{n}$ is the estimate of n , the tracked spread of V_f , and $\hat{n}_s$ is the estimate of n_s , the noise in V_f caused by collisions. $\hat{n}$ can be computed by dividing the summed spread of several starting layers (where the bitmaps are not almost full) by their proportion of the total spread, similar to [1]. Then the relative standard deviation of $\hat{n}$ is:

$$SD\left[\frac{\hat{n}}{n}\right] \lesssim \frac{\sqrt{\frac{1}{2}(e^\rho + e^{\rho/2} - 2) + e^{\rho/4} - 1}}{\rho\sqrt{2}s}} \approx \frac{0.7979}{\sqrt{s}}. \quad (1)$$

Considering that n_s and $\hat{n}_s$ are defined as the average noise contributed by numerous small flows, we figure out that regardless of whether f is a large flow, its impact on $\hat{n}_s$ is always marginal. Thus, combining the assumption that the noise is evenly distributed in m columns, we can approximate n_s by $n_s \approx \frac{s \times n_{s,m}}{m}$, where $n_{s,m}$ denotes the total spread of all small flows in M . Considering that $\hat{n}$ can be computed by adding up the inverse of p_M (Equation (2)), which makes the computation identical to a martingale PCSA [2], we have $SD\left[\frac{n_{s,m}}{n_{s,m}}\right] = \frac{0.5916}{\sqrt{m}}$ since each column in M is identical to a

PCSA word. Because $n_s \approx \frac{s \times n_{s,m}}{m}$, we have $SD[\frac{\hat{n}_s}{n_s}] = \frac{0.5916}{\sqrt{m}}$. Since $SD[\hat{n} - \hat{n}_s] = \sqrt{Var[\hat{n}] + Var[\hat{n}_s] - 2Cov(\hat{n}, \hat{n}_s)}$, we have:

$$SD[\frac{\hat{n}_f}{n_f}] = \frac{\sqrt{Var[\hat{n}] + Var[\hat{n}_s] - 2Cov(\hat{n}, \hat{n}_s)}}{n_f}. \quad (2)$$

Because $\hat{n}$ and the martingale estimate $\hat{n}_s$ are unbiased estimates, i.e., $E[\frac{\hat{n}}{n}] = 1$ and $E[\frac{\hat{n}_s}{n_s}] = 1$, we can compute the variance of $\hat{n}$ and $\hat{n}_s$ by $Var[\hat{n}] = n^2 SD[\frac{\hat{n}}{n}]^2$ and $Var[\hat{n}_s] = n_s^2 SD[\frac{\hat{n}_s}{n_s}]^2$. Because $Cov[\hat{n}, \hat{n}_s] = Cov[\hat{n}_f + \hat{n}_s, \hat{n}_s] = Cov[\hat{n}_f, \hat{n}_s] + Cov[\hat{n}_s, \hat{n}_s]$, where $\hat{n}_f$ and $\hat{n}_s$ are approximately independent as analyzed above, we have $Cov[\hat{n}, \hat{n}_s] \approx Var[\hat{n}_s]$. Then we can obtain the closed-form approximation of $SD[\frac{\hat{n}_f}{n_f}]$:

$$\begin{aligned} SD[\frac{\hat{n}_f}{n_f}] &\approx \frac{\sqrt{Var[\hat{n}] - Var[\hat{n}_s]}}{n - n_s} \\ &\approx \frac{\sqrt{0.636n_f^2 + 1.273n_f \times n_s + (0.636 - 0.35\frac{s}{m})n_s^2}}{n_f \times \sqrt{s}}. \end{aligned} \quad (3)$$

Considering that $n_s \leq \frac{n_M \times s}{m}$ where n_M is the total spread of all flows, and $m \gg s$, we can substitute this upper bound for n_s and omit $\frac{s}{m}$ in Equation 2, obtaining the following upper error bound:

$$SD[\frac{\hat{n}_f}{n_f}] \leq \frac{0.7979\sqrt{\frac{n_f^2}{s} + \frac{2n_f \times n_M}{m} + \frac{n_M^2 \times s}{m^2}}}{n_f}. \quad (4)$$

We make several approximations, particularly in the upper-bound analysis used to derive $SD[\frac{\hat{n}_s}{n_s}]$ and $Cov[\hat{n}, \hat{n}_s]$. The resulting standard deviation incorporates these approximations.

C.2 Asymptotic Unbiasedness of the Heavy Part

The estimation results of the heavy part are determined by two components: the virtual martingale estimator and the probabilistic replacement strategy.

Let f_i be the i -th largest flow, whose actual spread is λ . The estimated flow spread of f_i in the heavy part is:

$$C_{f_i} = E_\lambda - X_i + Y_i, \quad (5)$$

where E_λ is the martingale estimator that computes the sum of spread updates of f_i , X_i is the downward-biased estimation error caused by f_i being replaced by other flows, Y_i is the estimation error caused when f_i replaces other flows.

Let S denote the state space of f_i 's V2B, where we only consider the g feasible layers starting from the γ -th layer. We model the state evolution of the V2B as a Markov chain $(S_1, S'_1, S_2, S'_2, \dots, S_\lambda)$, where S_j is the state after seeing j distinct elements of f_i and S'_j is the state just before seeing the $(j+1)$ -th distinct element of f_i , because elements from

other flows can be hashed to the V2B between two adjacent elements of f_i . Define $P_j = \Pr(S_j \neq S'_{j-1} | S'_{j-1})$ to be the precise state change probability, which depends only on S'_{j-1} . And let P'_j be the value of $p(n, s, g)$, the expected state change probability of g feasible layers computed according to Equation (2). According to our virtual martingale mechanism, E_λ can be computed by

$$E_\lambda = \sum_{j=1}^{\lambda} \mathbb{1}\{S_j \neq S'_{j-1}\} \times \frac{1}{P'_j}. \quad (6)$$

Considering that the g feasible layers are all s -bit bitmaps that have higher estimation accuracy as s increases [1], and estimation accuracy is proportional to the accuracy of the bitmap change probability (or the fraction of zero bits), we derive that when $s \rightarrow \infty$, the actual change probability and the expected change probability (as used in Equation (2)) are asymptotically equal. Further, P_j and P'_j are also asymptotically equal if the spread value C_{f_i} is a good approximation of n_{f_i} . Then, by the linearity of expectation and the law of total expectation, we have

$$\begin{aligned} \mathbb{E}(E_j) &= \mathbb{E}(\mathbb{E}(E_{j-1} | S'_{j-1})) + \mathbb{E}(\Pr(S_j \neq S'_{j-1}) \cdot \frac{1}{P'_j} | S'_{j-1}) \\ &= \mathbb{E}(E_{j-1} | S'_{j-1}) + \mathbb{E}(P_j \cdot \frac{1}{P'_j}) \end{aligned} \quad (7)$$

Since C_{f_i} equals n_{f_i} in the initial state, when $s \rightarrow \infty$ such that $P_j \approx P'_j$, we can derive that

$$\mathbb{E}(E_j) \approx \mathbb{E}(E_{j-1}) + 1 \approx \mathbb{E}(E_{j-2}) + 2 \approx \dots \approx j. \quad (8)$$

For the downward-biased estimation error X_i , consider the following scenario: at time t , flow f has the smallest recorded flow spread $\hat{n}_f^{(t)}$ in the bucket. Let $r^{(t)} > 1$ denote the number of distinct elements from other flows attempting to replace flow f before the next distinct element of flow f . $X^{(t)}$ represents the downward biased estimation error added to $\hat{n}^{(t)}$. If all $r^{(t)}$ replacement attempts fail, flow f retains its recorded size $\hat{n}^{(t)}$ unchanged, implying $X^{(t)} = 0$. However, if flow f is successfully replaced by another flow, it loses its recorded spread, i.e., $X^{(t)} = \hat{n}^{(t)}$. The probability that all replacement attempts fail is given by $(1 - \frac{1}{\hat{n}^{(t)}})^{r^{(t)}}$. Therefore, the expected value of $X^{(t)}$, denoted as $\mathbb{E}(X^{(t)})$, is:

$$\mathbb{E}(X^{(t)}) = \hat{n}^{(t)} \left[1 - \left(1 - \frac{1}{\hat{n}^{(t)}} \right)^{r^{(t)}} \right] \leq r^{(t)}. \quad (9)$$

The inequality in the above formula can be easily proved by differentiation. Assuming $r^{(t)}$ is independent of $\hat{n}^{(t)}$, we have:

$$\mathbb{E}(X) = \sum_t \mathbb{E}(X^{(t)}) \leq \sum_t r^{(t)} = r. \quad (10)$$

If we choose an appropriate bucket array size m' such that the number of distinct elements trying to replace f , denoted by n_r , is negligible compared to n_f , then $\mathbb{E}(\frac{X_i}{n_f}) \leq \frac{n_r}{n_f} \approx 0$.

For Y_i , we can prove that when a flow f tries to replace another flow, the estimation error on its flow spread caused by the replacements, *i.e.*, Y_i , is unbiased. Let f' be the flow with the smallest recorded spread $n_{\min}$ in the bucket. Once flow f triggers a state change event (with probability P_j) and tries to replace flow f' , the estimated flow spread of f will be incremented by $n_{\min}$ with probability $\frac{1}{n_{\min}}$. Therefore, the expected increment to the spread of flow f is 1, which proves the unbiasedness.

Finally, by following the above derivations, we obtain that

$$\mathbb{E}(\frac{C_{f_i}}{n_{f_i}}) = \mathbb{E}(\frac{E_{\lambda}}{n_{f_i}}) - \mathbb{E}(\frac{X_i}{n_{f_i}}) + \mathbb{E}(\frac{Y_i}{n_{f_i}}) \approx 1, \quad (11)$$

when $s \rightarrow \infty$ and $m' \rightarrow \infty$. Therefore, the estimates produced by the heavy part are asymptotically unbiased.

C.3 Time Complexity of Per-packet Processing

The processing of every flow element requires two hash operations to determine one bit from the MV2B. If the bit is 0, our algorithm will set the bit to 1 and hash the flow to one bucket within the heavy part. Depending on the state of the bucket, there are three cases to consider, each requiring computation of the virtual martingale estimate and an update to (or replacement of) the flow record. Therefore, the time complexity of the per-packet operation is $O(1)$ since such an operation requires, at most, three hash operations, two memory accesses, and a few computations.

C.4 Validation of Error Bound

To validate our error bounds, we ran the per-flow spread measurement 1000 times using the same parameters, each time with a different hash seed. We then computed each flow's experimental relative standard deviation over its 1000 estimates and compared it to our theoretical error bound. The results confirm that the theoretical bound closely matches the experimental error. Table 1 provides an example of six representative flows in the CAIDA-2016 dataset along with their theoretical bounds and experimental errors, obtained using a total memory of 512KB (with a 3 : 10 ratio for MV2B's space allocation, $s = 128$, and $l = 16$).

D Sensitivity Analysis

We conduct sensitivity analysis on US's parameter settings, evaluating the accuracy of per-flow spread estimation and top- k super-spreader detection ($k = 300$).

Impact of $M_1 : M_2$ (MV2B size : heavy part size): We vary the memory size M from 0.5MB to 2.5MB. As depicted in Figure 2(a), an increase in M_1 reduces hash collisions, enabling the virtual martingale estimator to work better, which enhances the accuracy of per-flow spread estimation. When the

proportion of M_1 surpasses a certain threshold, the estimation accuracy will decrease because the number of buckets decreases, resulting in many flows not accurately recorded by the buckets. As shown in Figure 2(b), M_1 significantly affects super-spreader detection when the memory is small (such as 0.5MB) and has a smaller impact when the memory is larger (such as 1.5MB and above). Even if M_1 only occupies 10% of the memory, the F₁-Score for top- k super-spreader detection is not less than 91%.

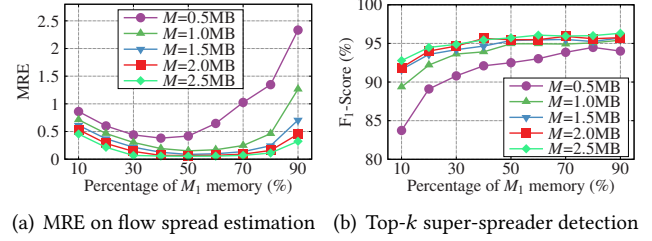


Figure 2. Impact of Parameter Setting on $M_1 : M_2$.

Impact of l (the number of layers in MV2B): As l increases from 16 to 32, MRE grows accordingly. Taking the estimation result of US at 2MB as an example, when l increases, the MRE rises from 0.09 to 0.11. This decline in precision is attributed to the fact that the greater the number of layers l , the smaller the number of columns m , resulting in more severe hash conflicts. The per-flow spread estimates directly affect the accuracy of super-spreader detection, so the detection accuracy decreases as well. Taking the detection results with a memory size of 2, MB as an example, the F₁-Score for top- k super-spreaders drops from 95.91% to 94.98%.

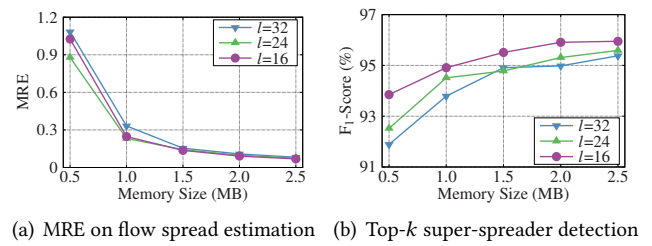
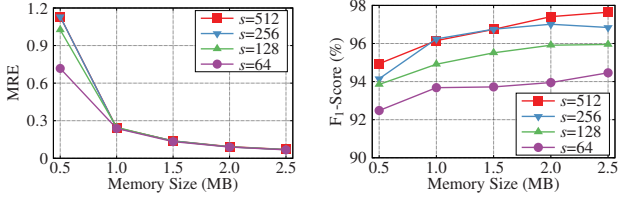


Figure 3. Impact of Parameter Setting on l .

Impact of s (the bitmap size in V2B): The results reveal that the impact of parameter s on accuracy is more pronounced when the memory size is relatively small. MRE tends to increase as s grows because a larger s leads to more sharing among flows, complicating the noise distribution. In the super-spreader detection task, as s increases, the F₁-Score also tends to increase because super-spreader detection focuses on large flows whose estimation results are more accurate when s is larger.

Table 1. Validation of Error Bound for MV2B.

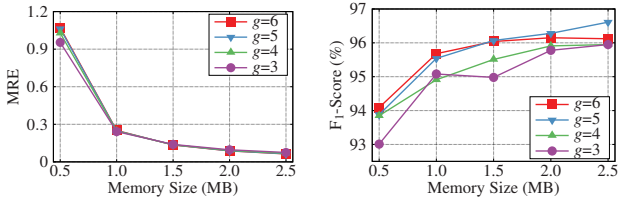
Flows	#1	#2	#3	#4	#5	#6
Actual Spread	512	947	2129	2704	4405	5161
Theoretical Error	0.1577	0.1176	0.0914	0.0870	0.0806	0.0791
Empirical Error	0.1369	0.1034	0.0800	0.0854	0.0775	0.0717



(a) MRE on flow spread estimation (b) Top- k super-spreader detection

Figure 4. Impact of Parameter Setting on s .

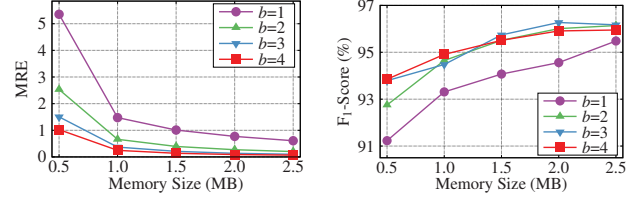
Impact of g (the number of layers involved in approximating p_f): The results demonstrate that the spread estimation accuracy slightly improves as g increases. Taking the results with 2.5 MB of memory as an example, the difference in MRE across different values of g is 0.01, while the difference in F_1 -Score is 0.66%. These differences are relatively small, demonstrating that while g does contribute to enhancing estimation accuracy, its impact is limited.



(a) MRE on flow spread estimation (b) Top- k super-spreader detection

Figure 5. Impact of Parameter Setting on g .

Impact of b (the number of slots in each bucket): As is shown in Figure 6(a), b has a considerable impact on per-flow estimation results. When the memory size M is 0.5MB, the MRE decreases from 5.35 to 1.03. The estimation accuracy with $b = 1$ is notably worse than that with other values of b . This is mainly due to the fact that multiple large flows can be mapped to the same bucket, but each bucket can only accurately record one of them with one slot ($b = 1$), and other flows must obtain their estimates from the MV2B structure. Similarly, the smaller b is, the lower the accuracy of super-spreader detection, because fewer slots cause more actual super-spreaders to be lost.



(a) MRE on flow spread estimation (b) Top- k super-spreader detection

Figure 6. Impact of Parameter Setting on b .

References

- [1] Cristian Estan, George Varghese, and Mike Fisk. 2003. Bitmap Algorithms for Counting Active Flows on High Speed Links. In *Proceedings of the 3rd ACM SIGCOMM Conference on Internet Measurement (IMC '03)*. Association for Computing Machinery, 153–166.
- [2] Seth Pettie, Dingyu Wang, and Longhui Yin. 2021. Non-Mergeable Sketching for Cardinality Estimation. In *48th International Colloquium on Automata, Languages, and Programming (ICALP 2021) (Leibniz International Proceedings in Informatics (LIPIcs))*, Nikhil Bansal, Emanuela Merelli, and James Worrell (Eds.), Vol. 198. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 104:1–104:20. <https://doi.org/10.4230/LIPIcs.ICALP.2021.104>